

AI/ML Internship – Day 68 Notes & Tasks

Module 9: Sentence Embeddings & Sentence Transformers – Converting Meaning into Numbers

Part 1: Recap of Day 67

Yesterday we learned:

-  PDF Text Extraction
-  Document Chunking
-  Vector Database
-  Similarity Search
-  RAG Pipeline

Today we will learn:

- What are Embeddings?
 - Why Embeddings are important
 - Sentence Transformers
 - Semantic Similarity
 - Creating Embeddings using Python
-

Part 2: What is an Embedding?

Definition

An **Embedding** is a numerical representation (vector) of text, images, or other data that captures its meaning.

Instead of storing words as plain text, AI converts them into numbers.

Simple Definition

Embeddings are **AI's way of understanding language using numbers**.

Example:

Word:

Artificial Intelligence



Embedding:

[0.42, -0.18, 0.91, 0.34, 0.77, ...]

These numbers represent the meaning of the text.

📌 Part 3: Why Do We Need Embeddings?

Computers cannot understand language directly.

Example:

I love AI.

For humans, this has meaning.

For a computer, it is just characters.

Embeddings convert text into numbers so that the computer can compare meanings.

📌 Part 4: Keyword Search vs Semantic Search

Suppose the document contains:

Artificial Intelligence is transforming education.

User asks:

How is AI changing schools?

Keyword Search

✗ May fail because the exact words are different.

Semantic Search (Embeddings)

✓ Understands that:

- AI = Artificial Intelligence
- Schools = Education

Returns the correct document.

Part 5: What is a Sentence Transformer?

A **Sentence Transformer** is a pre-trained AI model that converts sentences into embeddings. It understands the meaning of complete sentences.

Popular models include:

- all-MiniLM-L6-v2
- all-mpnet-base-v2
- multi-qa-MiniLM
- paraphrase-MiniLM-L6-v2

Part 6: Installing Sentence Transformers

Install the library:

```
pip install sentence-transformers
```

Import it:

```
from sentence_transformers import SentenceTransformer
```

Part 7: Loading a Model

```
from sentence_transformers import SentenceTransformer
```

```
model = SentenceTransformer(  
    "all-MiniLM-L6-v2"  
)
```

This downloads a pre-trained embedding model.

Part 8: Creating Embeddings

```
from sentence_transformers import SentenceTransformer
```

```
model = SentenceTransformer(  
    "all-MiniLM-L6-v2"  
)
```

```
"all-MiniLM-L6-v2"
)

sentence = "Artificial Intelligence is changing the world."

embedding = model.encode(sentence)

print(embedding)

Output (example):
[0.25, -0.74, 0.16, 0.81, ...]
```

Part 9: Multiple Sentence Embeddings

```
sentences = [
    "Artificial Intelligence",
    "Machine Learning",
    "Python Programming"
]

embeddings = model.encode(sentences)
```

```
print(embeddings)
```

Each sentence gets its own embedding vector.

Part 10: Semantic Similarity

Suppose we have:

Sentence 1:

I love Artificial Intelligence.

Sentence 2:

AI is my favourite subject.

Sentence 3:

I like playing football.

Sentence 1 and Sentence 2 have similar meanings.

Sentence 3 is unrelated.

Embeddings help AI understand this difference.

Part 11: Real-Life Example

Knowledge Base:

Course Duration: 3 Months

Course Fee: ₹5000

Location: Technopark

User asks:

How long is the internship?

Even though the document says **Duration** and the user says **How long**, embeddings identify the similarity and retrieve the correct chunk.

Part 12: Embedding Workflow

Sentence

|



Sentence Transformer

|



Embedding Vector

|



Store in Vector Database



Similarity Search



Retrieve Best Match

Part 13: Real-World Applications

Search Engines

Google uses semantic search techniques.

ChatGPT

Converts prompts into embeddings internally.

Recommendation Systems

Netflix

YouTube

Spotify

Recommend similar content.

Document Search

Find similar documents even if the wording is different.

RAG Systems

Retrieve relevant document chunks.

Part 14: Advantages

- ✓ Understands meaning instead of exact words
 - ✓ Improves search accuracy
 - ✓ Handles synonyms
 - ✓ Essential for RAG
 - ✓ Supports semantic search
-

Part 15: Limitations

- ✗ Creating embeddings takes processing time
 - ✗ Large datasets require more storage
 - ✗ Model quality affects search accuracy
 - ✗ Different models may produce different results
-

Part 16: Theory Questions

What is an embedding?

An embedding is a numerical vector representation of text that captures its semantic meaning.

Why are embeddings used?

Embeddings allow computers to compare the meaning of text rather than relying only on exact keyword matches.

What is a Sentence Transformer?

A Sentence Transformer is a pre-trained AI model that converts sentences into embedding vectors.

What is semantic search?

Semantic search retrieves information based on meaning rather than exact word matching.

Give three applications of embeddings.

- RAG systems
 - Search engines
 - Recommendation systems
-

Part 17: Interview Questions

What is the difference between keyword search and semantic search?

Keyword search relies on exact word matches, while semantic search retrieves information based on meaning.

Why are embeddings important in RAG?

Embeddings help retrieve the most relevant document chunks based on semantic similarity.

What is Sentence Transformers?

It is a Python library that generates high-quality sentence embeddings using pre-trained transformer models.

Name two popular Sentence Transformer models.

- all-MiniLM-L6-v2
 - all-mpnet-base-v2
-

Why do vector databases store embeddings instead of plain text?

Because vector databases perform fast similarity searches using numerical vectors.

Tasks for Day 68

Theory

1. Define an embedding.

2. Explain semantic search.
 3. What is a Sentence Transformer?
 4. Differentiate keyword search and semantic search.
 5. Why are embeddings important in RAG?
-

Practical Task 1

Install the Sentence Transformers library.

```
pip install sentence-transformers
```

Practical Task 2

Load the **all-MiniLM-L6-v2** model.

Practical Task 3

Generate embeddings for these sentences:

- Artificial Intelligence
 - Machine Learning
 - Deep Learning
 - Python Programming
-

Practical Task 4

Research three popular embedding models and compare their features.

Practical Task 5

Draw the embedding generation workflow diagram.